

IDATT1005 Systemutvikling 1

Eksamensøving

MMI

Menneske maskin interaksjon

Interaksjonsdesign

- Hva er definisjonen på interaksjonsdesign?
- Hva er fokuset med interaksjonsdesign?
- Hva er målet med interaksjonsdesign?
- Hva er en modell når man jobber med IxD?
- Hva er prosessen ved interaksjonsdesign?

- Hva er de fem dimensjonene i IxD?
- Hva er Don Normans prinsipper om interaksjonsdesign?

MMI

Menneske maskin interaksjon

Definisjon: utforme hvordan brukere samhandler med digitale produkter. Skape intuitive, effektive og meningsfulle opplevelser for brukerne.

Fokus: å forstå brukerens behov, mål og kontekst og designe UI og interaksjonsmønstre som passer

Mål: skape brukeropplevelser som er intuitive, effektive, tilfredstillende og engasjerende.

Modell når man jobber med IxD: Er en iterativ prosess lik moderne utviklingsmetoder

Prosess for interaksjonsdesign

1. Definer krav
2. Design alternativer
3. Lag prototype
4. Evaluer

De ulike dimensjonene som man bruker til å kommunisere med en bruker:

1. **Ord** - forståelig og tilgjengelig
2. **Visuell representasjon** - grafiske elementene
3. **Fysiske objekter og rom**

4. **Tid** - brukt på de første 3 dimensjonene
5. **Oppførsel** - følelsene og reaksjonene brukeren får når de samhandler med produktet

Don Normans prinsipper om interaksjonsdesign:

1. **Synlighet på funksjoner** - vise at de er til stede og hvordan de skal brukes. Brukeren burde gjenkjenne lett hvilke funksjoner som er tilgjengelig. Motsatt; funksjoner som ikke er nødvendig til gitt tid burde ikke være synlig
2. **Feedback**: signaliserer til brukeren hva som foregår gjennom. Gjøres gjennom lyder, visualiseringer, fysisk osv.
3. **Begrensninger**: begrenser brukeren ulike former for interaksjon som er mulig å gjøre i et gitt øyeblikk. Gjør det lettere å lede brukeren gjennom prosessen og gjøre applikasjonen mer brukervennlig
4. **Avbildning**: Konsept til hvordan en funksjon irl oversettes til interaksjonsmuligheter i applikasjonen. Burde en stryken på lys justert med knapp, hjul, slider elns.
5. **Konsistens**: lage ting slikt at brukeren gjør lignende ting på lignende måter flere steder i applikasjonen.
6. **Tilbydelser**: hvilke handlinger en bruker *opplever* at hun kan gjøre med et system. Henger med hvordan systemet er laget og brukerens erfaring med andre systemer.

Prinsippene ment for å diskutere og beskrive brukskvaliteten til UI, og ikke som en sjekkliste for interaksjonsdesign.

Prototyping

- Hva er prototyping?
- Hvorfor prototyper vi?
- Hvilke dimensjoner finnes det ved prototyping?
- Hva er de ulike klassene av prototyping, og hva er deres kompromisser?
- Hva er de ulike aspektene ved utvikling av prototyper?

MMI

Menneske maskin interaksjon

Dimensjoner ved prototyping:

1. **Utseende**
2. **Innhold**
3. **Funksjonalitet**
4. **Interaktivitet**
5. **romlig struktur**

Klasser og kompromisser:

- **Horisontal prototyper:** lite dypde og brukes til høynivåpresentasjoner
- **Vertikal prototype:** mye dypde

Aspekter ved utvikling av prototyper:

- **Evolusjonære prototyper**
- **Bruk og kast**
- **Lavoppløslige prototyper:** lite interaksjon
- **Høyoppløslige prototyper:** mye interaksjon

Evaluering

- Hva er evaluering?
- Hva er forskjellen mellom testing og evaluering?
- Hva er de ulike evalueringsformene?
- Hva er de ulike evalueringene som involverer brukere?
- Hva er de ulike evalueringene som *ikke* involverer brukere?
- Hva er de ulike evalueringsmetodene?
- Hva er de ulike evalueringsteknikkene?

MMI

Menneske maskin interaksjon

Er å finne verdien av noe. Bredt begrep som inkluderer: tester, formelle modeller, gjennomgang, prototyper

Testing vs Evaluering:

- **Testing:** Avdekke feil (verifisering – *are we building the product right?*)
- **Evaluering:** Vurdere nytteverdi/brukeropplevelse (validering – *are we building the right product?*). Inkluderer også testing.

Alt som kan gi brukeren en idé om den endelige løsningen kan evalueres.

Evalueringsformer:

- **Formativ:** man gjør en evaluering for å påvirke utviklingen og sikre at designende man jobber med oppfyller kravene, utføres underveis.
- **Summativ:** evaluering av sluttproduktet

Evaluering som involverer brukere

- **Brukertester** - lar en bruker bruke produktet i kontrollerte omgivelser og observer hva som blir brukt, feil, interaksjoner osv.
- **A/B tester** - lager to eller flere varianter av et design, presenterer brukere for en tilfeldig variant og ser hvilken variant som presterer best

- **Feltstudier** - naturlige omgivelser
- **Opportunistiske evaluering** - en bruker gi tilbakemelding på en spesifikk ide eller problemstilling

Evaluering som ikke involverer brukere

- **Heuristiske gjennomganger** - evaluer produkt gjennom regler og retningslinjer for brukerkvalitet
- **Pluralistiske gjennomganger** - brukere, utviklere og brukerkvalitetseksperter jobber sammen gjennom viktige scenarioer
- **Prediktive modeller** - maskinlæringsmodeller som prøver å forutsi brukeropplevelse

Evalueringsmetoder (*Metoder*):

- **Observasjon**
- **Spørre brukere**
- **Direkte testing**
- **Spørre eksperter**
- **modellere**

Evalueringsteknikker (*Metoder for data*):

- **Kodevalidering**
- **Testing i forskjellige nettleser og på forskjellige plattformer**
- **Testing av fleksibiliteten til et design**
- **Testing av grad av universell utfordring og brukerkvalitet**
- **Testing av feilmeldinger**

Brukertesting

- Hva er brukertesting?
- Hva er definisjonen på brukertesting?
- Hva er målet med brukertesting?
- Hva bør brukertesting ha?

MMI

Menneske maskin interaksjon

Definisjon: Simulerer en reell situasjon der testing inkluderer konkrete oppgaver. Observerer brukeren og brukes til å evaluere brukervennligheten.

Mål: få innsikt man trenger for å gjøre produktet enklere å bruke

Brukertest burde ha spesifikke og begrenset hensikt. Problem -> formål med brukertest

Universell utforming

- Hva er definisjonen på universell utforming?
- Hva er hensikten med universell utforming?
- Hva handler universell utforming *ikke* om?
- Hva bidrar universell utforming til?

- Hva er prinsippene for universell utforming?

MMI

Menneske maskin interaksjon

Definisjon: Går ut på at løsninger skal være utformet slik at de kan benyttes av flest mulig uten å diskriminere.

Hensikt: Produktet kan brukes av alle typer brukere uavhengig av fysisk begrensninger og kunnskapsnivå.

Handler ikke om: å lage tilleggsløsninger for de med spesielle behov, men lage ting som i utgangspunktet kan brukes av alle

Bidrar til: likeverdig deltakelse i samfunnet og øker brukervennligheten for alle og samtidig reduserer behov for individuelle tilpasninger.

Prinsipper for universell utforming (Central for Universal Design):

1. **Enkel og intuitiv i bruk:** Utformingen skal være lett å forstå uten hensyn til brukerens erfaring, kunnskap, språkferdigheter eller konsentrasjonsnivå.
2. **Forståelig informasjon:** Utformingen skal kommunisere nødvendig informasjon til brukeren på en effektiv måte.
3. **Toleranse for feil:** Utformingen skal minimalisere farer og skader som kan gi ugunstige konsekvenser, eller minimalisere utilsiktede handlinger.
4. **Like muligheter for alle:** Utformingen skal være brukbar og tilgjengelig for personer med ulike ferdigheter.
5. **Fleksibel i bruk:** Uansett individuelle preferanser og ferdigheter. Den

synshemmede skal kunne høre, den hørselhemmede se og så videre.

- 6. Lav fysisk anstrengelse:** Utformingen skal kunne brukes effektivt og bekvemt med minimum besvær.
- 7. Størrelse og plass for tilgang og bruk:** Hensiktsmessig størrelse og plass skal muliggjøre:
 1. Tilgang
 2. Rekkevidde
 3. Betjening og bruk, uavhengig av brukeren kroppsstørrelse, kroppsstilling og mobilitet.

Diskriminerings- og tilgjengelighetsloven stiller krav om universell utforming til offentlige og private virksomheter

Lovkrav til universell utforming

- Hvilke paragrafer omfatter likestilling og diskriminering?
- Hva sier de ulike paragrafene konkret?
- Når trådte disse paragrafene i kraft?
- Hva er forskriften om universell utforming av IKT?
- Når trådte forskriften om universell utforming av IKT i kraft?
- Hva er WAD?
- Når kom de nyeste kravene fra WAD?
- Hvilke gode argumenter finnes det for UU?

MMI

Menneske maskin interaksjon

Lov om likestilling og forbud mot diskriminering (§18, §19a, §41): trådte i kraft 1. januar 2018.

§17: Offentlige og private virksomheter rettet mot allmennheten har plikt til universell utforming av alminnelige funksjoner

§18: IKT-løsninger som underbygger virksomhetens alminnelige funksjoner skal være universelt utformet

§41: Eksisterende IKT-løsninger skal være universelt utformet fra 1. januar 2021

Forskrift om universell utforming av IKT: trådte i kraft 1. juli 2013

EUs Webdirektiv (WAD): oppdatert regelverk, nye krav gjaldt fra 1. februar 2023

Gode argumenter for UU:

Mer inkluderende (nesten 1 million opplever utfordringer på nettet)

Enklere å bruke (god informasjonsstruktur, enkel navigasjon)

Mer mobiltilpasset (store knapper, god kontrast fungerer på mobilskjerm)

Mer synlig for Google (god kodenstandard = bedre SEO)

Mer lønnsomt (økt omsetning, færre henvendelser til support)

WCAG 2.0

- Hva er WCAG 2.0?
- Hva er målet med WCAG 2.0?
- Hva er noen krav til WCAG 2.0?
- Hva er prinsippene i det øverste nivået av WCAG 2.0?
- Hva er det nettsider må oppfylle for å følge WCAG 2.0?

MMI

Menneske maskin interaksjon

Definisjon: Retningslinjer som ser utviklet for å hjelpe utviklere til å lage nettsteder som er tilgjengelig for alle, inkl personer med funksjonsnedsettelse.

Mål: sikre at folk, uavhengig av faktorer som kjønn, alder, og funksjonshemninger får tilgang til den samme informasjonen.

Noen krav:

Alt av innhold skal være tilgjengelig via tastaturnavigering

Bilder skal ha alt-funksjon

Video skal ha teksting eller alternativ lydspor

Formater skal være lett å forstå

Innhold skal være lett å forstå

Øverste nivå i WCAG retningslinjer er det fire prinsipper:

Mulig å oppfatte

Mulig å betjene

Forståelig

Robusthet

12 retningslinjer på nivået under som deles på 61 kriterier som nettsider må

oppfylle. ex: kontrastbruken i tekst som må vise kontrastforhold med bakgrunnen slik at fargeblinde og svaksynte kan lese

UML

Unified Modeling Language

UML

- Hvorfor UML?
- Hvordan fungerer UML?
- Hvilke ulike modeller finnes?

UML

Unified Modeling Language

Visualiserer konsept på en mer oversiktlig måte. Kan brukes der og da og for dokumentasjon etter.

Uttrykker mye informasjon med en modell

Use case

- Hva er et use case diagram?
- Hvordan lager man et use case diagram?

UML

Unified Modeling Language

Definisjon: Illustrerer overfladisk hvordan brukeren bruker et system. Gjøres ved å illustrere ulike scenarioer en bruker kan møte og hvordan brukeren kan navigere seg gjennom systemet.

Avklarer enkelte funksjonelle krav

Hvordan lager man en use case:

Har en aktør som representerer en bruker. Aktøren har assosiasjoner med flere use-caser koblet med en linje. Bruker "extend" og "include" for å koble sammen deler av. Kan også bruke systemgrense for å skille ulike use-caser i use-casen.

Domenemodell

- Hva er en domenemodell?
- Hva er hensikten med en domenemodell?
- Hva er viktig med et domenemodell?

UML

Unified Modeling Language

Definisjon: Beskrive data og funksjonalitet innad et domene abstrakt. Definerer en rekke konsepter og relasjonene mellom dem.

Mål: *alle* skal få felles forståelse over domenet

Domenemodellen skal ikke være detaljert som de andre modellene. Konseptene og relasjonene skal vises på en oversiktlig og enkelt måte. Ofte brukes et felles språk med navngiving osv.

Sekvensdiagram

- Hva er et sekvensdiagram?
- Hva er hensikten med et sekvensdiagram?
- Hva representerer de ulike elementene i et sekvensdiagram?

UML
Unified Modeling Language

Viser hvordan klassene/komponenter i applikasjonen samarbeider over tid

Elementene i et sekvensdiagram:

- **Deltakerne (klasser, aktører, objekt, komponenter) er boksene**
- **Funksjoner er horisontale linjer som peker mot høyre**
- **Stiplet vertikal linje: deltakers levetid/tidslinje**
- **Solid rektangel på vertikale linjen: når en deltaker utfører en operasjon**
- **Ramme representerer løkker**

Aktivitetsdiagram

- Hva er et aktivitetsdiagram?
- Hva er hensikten med et aktivitetsdiagram?
- Hva representerer de ulike elementene i et aktivitetsdiagram?

UML
Unified Modeling Language

Viser hvilke rekkefølge aktiviteter utføres i, dens betingelser og tilhørende handlinger. Dermed ser vi hvilke handlinger som utføres asynkronisk og hva som er avhengig av hverandre.

Elementene i et aktivitetsdiagram:

- **Aksjoner representeres med bokser. Start og slutt vises med svart sirkel. Overganger mellom aksjoner vises med piler**
- **Noder:**
 - Initial node** (start)
 - Final node** (slutt)
 - Decision node** (betingelse)
 - Fork node** (parallell start)
 - Join node** (parallell slutt)
 - Merge node**

Klassediagram

- Hva er et klassediagram?
- Hva er hensikten med et klassediagram?
- Hva representerer de ulike elementene i et klassediagram?
- Når brukes et klassediagram, og hvorfor brukes det da?

UML
Unified Modeling Language

Modellerer den statiske strukturen til et system. Viser klassene (typer objekter), attributtene, metodene og relasjonene mellom dem. Kan brukes til å generere kode (forward engineering) eller utledes fra eksisterende kode (reverse engineering).

Elementene i et klassediagram:

- **Klasse:** navn, attributter og metoder
- **Relasjoner:**
 - Avhengighet (dependency):** klasse brukes av en annen, svak kobling
 - Assosiasjon:** strukturell kobling mellom to klasser, kan ha multiplisitet
 - Aggregering (aggregation):** "del av"-relasjon, delene har eget levetid
 - Komposisjon (composition):** sterk aggregering – delene ødelegges når helheten ødelegges
 - Arv (inheritance/generalization):** subklasse arver fra superklasse
 - Realisering (realization):** klasse implementerer et grensesnitt (interface)

Brukes tidlig: se use-diagram → sekvensdiagram → klassediagram før koding

Pakkediagram

- Hva er et pakkediagram?
- Hva er hensikten med et pakkediagram?
- Hvordan brukes et pakkediagram?

UML
Unified Modeling Language

Grupperer relaterte UML-elementer (diagrammer, klasser, andre pakker) i en hierarkisk struktur. Brukes til å vise høynivåarkitekturen til et system.

Vanlig i programmering for å organisere kildekode

Kan vise lagdelt arkitektur (klient–server–data) eller funksjonell gruppering

Tilstandsmaskindigram

- Hva er et tilstandsmaskindigram?
- Hva er hensikten med et tilstandsmaskindigram?
- Hva er viktig for et tilstandsmaskindigram?
- Hvilke hoved elementer består et tilstandsmaskindigram av?
- Hvilke andre UML-diagrammer samspiller tilstandsmaskindigrammer med?
- Hvordan samspiller tilstandsmaskindigrammer med andre UML-diagrammer?

UML
Unified Modeling Language

Viser de ulike tilstandene til et objekt eller system, og overgangene som fører til tilstandsendringer.

Hvert objekt i systemet kan endre tilstand i sin levetid (f.eks. bruker: ikke-innlogget → innlogget → utlogget)

Hovedelementer:

- Starttilstand
- Sluttilstand
- Tilstander (states)
- Overganger (transitions) med triggere/hendelser

Utfyller sekvens- og aktivitetsdiagrammer ved å vise systemet fra et tilstandsperspektiv

Brukerhistorier

- Hva er brukerhistorier?
- Hvem skriver vanligvis brukerhistorier?
- Hvilket format bruker brukerhistorier?
- Hvordan modelleres brukerhistorier?

UML
Unified Modeling Language

Minste enhet av arbeid, vanligvis skrevet av produkteier.

Format: «As <> I want <> Such that <>»

Eksempel: «As <> I want <> Such that <>»

Modellering av brukerhistorier gjøres i UML via use case-diagrammer

Programvarearkitektur

Alt fra kildekoden til produktet

Programvarearkitektur

- Hva handler programvarearkitektur om?
- Hva er IEEE?
- Hva er definisjonen på programvarearkitektur i følge IEEE?

Programvarearkitektur
Alt fra kildekoden til produktet

Handler om å se den helhetlig strukturen til programmet, alt fra kildekoden til produktet.

IEEE: IEEE står for **Institute of Electrical and Electronics Engineers**. Det er verdens største tekniske profesjonsorganisasjon, og fungerer som en global autoritet som utvikler standarder og publiserer forskning innen teknologi, databehandling, kommunikasjon og elektronikk.

IEEE-definisjon: «Arkitektur er den fundamentale organiseringen av et programvaresystem, uttrykt gjennom dets komponenter, deres relasjoner til hverandre og til miljøet, samt prinsippene som styrer design og evolusjon.»

Hvorfor arkitektur?

- Hvorfor er programvarearkitektur viktig?

Programvarearkitektur
Alt fra kildekoden til produktet

Hvorfor?

- Forstå systemet tidlig – tidlige designbeslutninger og tidlig tilbakemelding gir:
 - Reduserte kostnader, unngår duplisering av ressurser
 - Økt kvalitet og hjelp til risikostyring
- Kommunikasjon mellom interessenter – gir felles og gjensidig forståelse
- Retningslinjer for utviklere gjennom hele prosjektet

Kompleksitetshåndtering

- Hva er kompleksitetshåndtering?
- Hva heter teknikken som brukes ved kompleksitetshåndtering?
- Hvorfor er kompleksitetshåndtering viktig?

Programvarearkitektur
Alt fra kildekoden til produktet

Håndtere kompleksitet ved å dele dem mindre domener/biter (Splitt og hersk teknikk).

Enklere delegere ansvarsområder i utviklingsteamet, samt isolere hvert del, som gjør koden mer testbar og robust

Bidrar til lav-kobling

Standalone application

- Hvordan er en standalone application delt opp?
- Hvilket bruksområde har en standalone application?
- Hvilke fordeler og ulemper er det ved en standalone application?

Programvarearkitektur
Modeller

Programvare som inneholder all logikk og fungerer på egenhånd
Funker bra for mindre prosjekter slik at kompleksitet blir unngått.

Klient - tjener arkitektur

- Hvordan er klient - tjener arkitektur delt opp?
- Hvordan fungerer en klient - tjener arkitektur?
- Hvilke fordeler og ulemper er det ved en klient - tjener arkitektur?

Programvarearkitektur
Modeller

Deler opp koden i to deler:

- (1) Kjører på klienten (presentasjon)
- (2) Kjører på serveren (logikk)

Serveren utfører oppgaver for de ulike klientene og klientene spør om de tjenestene de trenger.

Gjør at utviklere kan fokusere på deler av programmet. Fører til lav kobling som gjør det mer skalerbart.

Tre-lags-arkitektur

- Hvordan er en tre-lags-arkitektur delt opp?
- Hvordan fungerer tre-lags-arkitektur?
- Hvilke fordeler og ulemper er det ved tre-lags-arkitektur?

Programvarearkitektur
Modeller

Dele opp applikasjoner i 3 lag.

Presentasjonslag: Presentere data til brukeren vha UI-elementer. Håndterer interaksjoner og kaller på tjenester under.

Foretningslogikklag: Databehandling i applikasjonen, logikk og foretningsarbeid for brukeren

Data access lag: Gir tilgang til eksterne tjenester og APIer ved bruk av metoder for logikklaget slik at applikasjonen får utført arbeid. CRUD

Hvert lag utfører spesifikke oppgaver for et lag høyere. Hvert lag kan kun be om tjenester fra laget under eller over seg.

Gir lav kobling

Master-slave pattern

- Hvordan er en master-slave pattern delt opp?
- Hvordan fungerer master-slave pattern?
- Hvilke fordeler og ulemper er det ved master-slave pattern?
- Hvor brukes master-slave pattern?

Programvarearkitektur
Modeller

Bygd opp at to partier: master og slaver.

- Masteren: komponenten som delegerer arbeid til de forskjellige identiske slave komponentene.
- Slavene returnerer resultat til master.

Fordeler:

- Får et overordnet blikk over prosessen i master og kan gå over til implementeringen i slavene. Slavene kan enkelt enhetstestes mens mesteren kan integrasjonstestes opp mot slavene.

Brukes i databasereplikering:

- Master db fungerer som kilden og videresender sin tilstand til slavene, som synkroniserer seg til mesterens data. Kan da sende spørringer til hvilken som helst av databasene som gir systemet høyere kapasitet enn en enkel db.

Pipe-filter pattern

- Hvordan er pipe-filter pattern delt opp?
- Hvordan fungerer pipe-filter pattern?
- Hvilke fordeler og ulemper er det ved pipe-filter pattern?
- Hvor brukes pipe-filter pattern?

Programvarearkitektur
Modeller

Piper etterfulgt av rekke filter-nodes som prosesserer dataene til en *sink* som samler opp dataene.

Brukt til å strukturere systemer som produserer datastrømmer.

Broker pattern

- Hvordan er broker pattern delt opp?
- Hvordan fungerer broker pattern?
- Hvilke fordeler og ulemper er det ved broker pattern?
- Hvor brukes broker pattern?

Programvarearkitektur
Modeller

Hver komponent kommuniserer gjennom en broker, som fasiliteter kommunikasjon og koordinasjon. Serveren gir sine kapasiteter til brokeren og klienten spør etter service fra brokeren, og brokeren sender klienten til et passende service.

Brukt til å strukturere systemer der komponentene ikke har kjennskap til hverandre.

Peer to Peer pattern

- Hvordan er peer to peer pattern delt opp?
- Hvordan fungerer peer to peer pattern?
- Hvilke fordeler og ulemper er det ved peer to peer pattern?
- Hvor brukes peer to peer pattern?

Programvarearkitektur
Modeller

Komponentene kalt peers.

Peers fungerer nesten lik klient-tjener, men kan ta en dynamisk rolle både tjener og klient.

Peersene kan sende forespørsler til hverandre og samarbeider ofte.

Event-bus pattern

- Hvordan er event-bus pattern delt opp?
- Hvordan fungerer event-bus pattern?
- Hvilke fordeler og ulemper er det ved event-bus pattern?
- Hvor brukes event-bus pattern?

Programvarearkitektur
Modeller

Fire hovedkomponenter: event listener, event source, channel og event bus.

Kilden (source) sender en melding til en spesifikk channel på event bussen. Listener abonnerer til channelen og sier ifra til andre komponenter når en melding blir sendt på channelen de abonnerer på.

MVC

- Hvordan er MVC delt opp?
- Hvordan fungerer MVC?
- Hvilke fordeler og ulemper er det ved MVC?
- Hvor brukes MVC?

Programvarearkitektur
Modeller

Deler en interaktive applikasjon i tre deler:

- Model: hoved funksjonaliteten, alt av data som vises i applikasjonen samt data som er viktig for prosessering
- View: Viser fram modellene og andre data til brukeren gjennom GUI
- Controller: Styrer informasjonsflyten ved å ta input og andre interaksjon fra brukeren, henter/sender data til andre tjenester og manipulerer modellene slik at GUI blir oppdatert.

Fordeler

Deler opp komponenter som fører til høy kohesjon og gjenbrukbarhet

Microservice

- Hva er microservice basert på?
- Hvordan er microservice delt opp?
- Hvordan fungerer microservice?
- Hvilke fordeler og ulemper er det ved microservice?
- Hvor brukes microservice?

Programvarearkitektur
Modeller

Basert på *single responsibility principle*: samle det som endres av samme grunn, og skill det som endres av ulike grunner.

Deler opp programvare i selvstendige tjenester med én ansvarsoppgave, sin egen data og eget kjøremiljø.

Fordeler

- Skalerbart
- Modulær
- Fleksibel bruk av teknologi
- Tilgjengelig (feil i én tjeneste stopper ikke hele systemet)
- Muliggjør kontinuerlig levering (CI/CD), optimalisert for DevOps

Ulemper

- Kompleks debugging og logging på tvers av tjenester
- Krever automatisering
- Vanskelig å teste (distribuerte systemer)
- Datainkonsistens – data spredt over mange databaser
- Økt ressursforbruk

Monolittarkitektur

- Hvordan er monolittarkitektur delt opp?
- Hvordan fungerer monolittarkitektur ?
- Hvilke fordeler og ulemper er det ved monolittarkitektur ?
- Hvor brukes monolittarkitektur ?

Programvarearkitektur
Modeller

Tradisjonell, sammenslått modell der alle deler av programvaren er pakket inn i ett system. Selvinneholdt og tett koblet.

Fordeler

- Bedre gjennomstrømning enn modulære tilnærminger
- Enklere å teste og debugge i begynnelsen

Ulemper

- Vanskelig å vedlikeholde – endring i én del krever oppdatering av hele applikasjonen
- Lite skalerbar/utvidbar sammenlignet med modulære arkitekturer
- Tett koblet – én feil kan stoppe hele systemet

Plugin-basert arkitektur

- Hvordan er plugin-basert arkitektur delt opp?
- Hvordan fungerer plugin-basert arkitektur?
- Hvilke fordeler og ulemper er det ved plugin-basert arkitektur?
- Hvor brukes plugin-basert arkitektur? Gi noen eksempler

Programvarearkitektur
Modeller

Består av to hoveddeler: et **kjerne-system** og **plugin-moduler** (utvidelser/extensions).

Kjernen administrerer levetiden til tilkoblede plugins
Plugins kan legges til eller fjernes fra kjernen når som helst

Fordeler

- Lav kobling og høy isolasjon mellom funksjoner
- Fleksibelt og utvidbart

Ulemper

- Endringer i kjernen kan ødelegge alle plugins (bakoverkompatibilitet viktig)
- Mange plugins kan senke ytelsen
- Kompleks testing av hele systemet

Eksempler: Chrome Web Store, IDE-er (IntelliJ, VS Code, Eclipse)

Deployment-arkitekturer

- Hva er tradisjonell deployment?
- Hva er fordeler og ulemper med tradisjonell deployment?
- Hva er virtualisering (VMs)?
- Hva er fordeler og ulemper med virtualisering?
- Hva er container-deployment?
- Hvilke nøkkelteknologier bruker container-deployment, og hvordan?
- Hva er fordeler og ulemper med container-deployment?

Programvarearkitektur
Deployment

Tradisjonell deployment: Applikasjoner kjøres direkte på fysiske servere. Ressursallokering er et problem – én app kan ta opp for mye.

Virtualisering (VMs): Kjøre flere virtuelle maskiner på én fysisk server. Hvert VM har eget OS. Problem: ressurser reserveres selv om de ikke brukes.

Container-deployment: Containere deler OS med hverandre men er isolert. Lettere og mer effektive enn VMs.

Nøkkelteknologier:

- **Docker** (container-teknologi)
- **Kubernetes** (container-orkestrering).

Fordeler:

- Containere er enkle å distribuere og gir konsistent kjøremiljø ("fungerer likt overalt")
- Løst koblet, distribuert, elastisk – som microservices

Systemutviklingsmetodikk

Metoder

Før smidig metoder

- Hvilke metoder fantes før smidig metoder?
- Hvordan fungerer disse metodene?
- Hvorfor ble de valgt?
- Hvilke fordeler og ulemper har disse metodene?

Systemutviklingsmetodikk
Metodikk

Vannfallsmodellen:

Flere store faser gjennomført en og en, med krav først også design.
Mye dokument overlevert fase til fase og nye folk hver fase
Vanskelig å skalere og mange prosjekter feilet.

UP (Unified Process):

innførte iterasjoner for å takle vannfallsmodellen sine problemer rundt endringer underveis.
UML-modellering svært sentralt

Smidige metoder

- Hvorfor oppstod smidige metoder?
- Hvordan ble smidige metoder kjent?
- Hva er de grunnleggende prinsippene for smidig metode?
- Kort oppsummert, hva består manifestet av?

Systemutviklingsmetodikk
Metodikk

Oppsto som reaksjon mot den tunge vannfallsmodellene. Ble kjent når *The Agile Manifesto* ble utgitt.

Grunnleggende prinsipper: for smidig metode

1. **Problem based:** Større problemer brytes ned i mindre og prioriteres.
2. **Collaborative:** Flere personer er en del av prosessen fra begynnelsen.
3. **Iterativ:** Prosessen er iterativ
4. **Adaptiv:** fremgang er adaptiv og ikke lineær.

Manifesto kort oppsummert:

- **Individer og interaksjoner fremfor prosesser og verktøy:** menneskene involvert og deres samarbeid er den viktigste faktoren for prosjektet. Prioriterer kommunikasjon og samarbeid fremfor strenge prosesser og bruk av spesifikke verktøy.
- **Fungerende programvare fremfor omfattende dokumentasjon:** Målet er å skape verdi gjennom programvaren som utvikles.
- **Kundesamarbeid fremfor kontraktsforhandlinger:** Vektlegger samarbeid med kunden for å forstå deres behov og tilpasse endringer over tid enn å fokusere på detaljerte kontrakter og opprinnelige krav.
- **Reagere på endring fremfor å følge en plan:** Smidig metoder er

tilpasningsdyktig og fleksible. Å respondere effektivt på endrede krav og forutsetninger er viktigere enn å følge en forhåndsbestemt plan.

Extreme Programming

- Hva kjennetegner XP?
- Hvor mange regler har XP?
- Hva er noen av reglene for XP?
- Hva er de sentrale verdiene innen XP?

Systemutviklingsmetodikk
Smidige metoder

Har nesten ingen dokumentasjon og modellering.

Har 29 faste regler.

Noen regler:

- Alltid parprogrammering
- Skriv enhetstester før koding
- Integrer kun én liten endring av gangen
- Enkel, tydelig kode og hyppig refaktorering
- Ha kort feedback-loop, hyppige tester og tilbakemeldinger fra kunde skal bidra til kontinuerlige endringer.
- Ha korte iterasjoner med leveranser mellom dem

Fem sentrale verdier innen XP:

- **Communications**
- **Simplicity**
- **Feedback**
- **Courage**
- **Respect**

(Det trengs god kommunikasjon, koden skal simpel, tilbakemelding fra testene, teamet og kunden, man må ikke vær redd for refaktorisere dårlig kode, ha respekt for sitt og andres arbeid.)

Test-Driven development

- Hva er TDD?
- Hvilken sammenheng har TDD og XP?
- Hva legger TDD fokus på?
- Hvordan er tester og krav beskrevet med TDD, og hva er sammenhengen?
- Hvilke fordeler og ulemper har TDD?

Systemutviklingsmetodikk
Smidige metoder

Hva? Skriver tester for en bestemt funksjonalitet først, kjører testen for å se at den mislykkes og deretter skrive kode for at den skal bestå.

Videreføring av XP og har samme "regler" når det gjelder refaktorering av dårlig kode.

Alltid fokus på enhetstesting, noen ganger fokus på integrasjonstesting, sjeldent fokus på systemtesting.

Tester og krav er sammensmeltet:

Enhetstester setter krav til hva koden kan gjøre samtidig som den tester.

Fordeler

- Kodekvalitet
- Testdekning
- Rask tilbakemelding
- Økt tillit
- Testene fungerer også som en form for dokumentasjon.

Ulemper

- Lærekurve: krever endring i tenkemåte
- Tidsbruk å skrive tester
- Vedlikehold av testene

Behavior-Driven development

- Hva er BDD?
- Hvordan gjennomføres BDD?
- Hva sørger BDD for?
- Hvilke fordeler og ulemper har BDD?
- Hva er Gherkin format?

Systemutviklingsmetodikk
Smidige metoder

Hva? Behavior-driven development (BDD) er en programvareutviklingsmetode som knytter forretningsmål og teknologi sammen. Den fokuserer på å definere systemets atferd fra brukerens perspektiv, ved hjelp av lettfattelig «naturlig språk» som alle involverte forstår

Bruker *user-stories* til å definere kundekrav og testcaser for akseptansetester

Gruppa møtes for å komme med konkrete akseptkriterier i en brukerhistorie. Akseptkriterier skrevet vha domenespesifikt språk som UML og lagt inn i testscenario fil som konverteres til kjørbare tester, vanligvis automatisk. Teste applikasjonens oppførsel fra sluttbrukerens perspektiv

Sørger for at use cases fungerer på et høyere nivå

Gherkin format

Gherkin er et strukturert, menneskevennlig tekstformat som brukes til å beskrive programvarens oppførsel. Det fungerer som en bro mellom utviklere, testere og forretningsfolk, og brukes ofte i rammeverk som *Cucumber* for å automatisere programvaretesting

Behavior-driven development (BDD) er en programvareutviklingsmetode som knytter forretningsmål og teknologi sammen. Den fokuserer på å definere systemets atferd fra brukerens perspektiv, ved hjelp av lettfattelig «naturlig språk» som alle involverte forstår

Det magiske "Given-When-Then"-formatet

For å unngå misforståelser mellom forretningssiden (produktansvarlige, testere) og utviklere, skriver teamet krav og spesifikasjoner sammen. Dette gjøres i et standardisert format kalt **Gherkin**: []

Gitt (Given): Starttilstanden (konteksten).

Når (When): Handlingen brukeren utfører.

Så (Then): Det forventede resultatet.

Eksempel på en spesifikasjon:

- **Gitt** at en kunde har en vare i handlekurven og er innlogget
Når kunden velger å betale og fullfører kjøpet
Så skal ordren bli registrert som godkjent
Og kunden skal motta en bekreftelse på e-post

Nøkkelelementer i Gherkin

Gherkin-filer lagres med filendingen .feature og følger en streng oppbygging:

Feature: Beskriver funksjonaliteten som skal testes.

Scenario: Et spesifikt eksempel på hvordan funksjonen skal oppføre seg.

Given-When-Then: Logikken som driver testene:

Given (Gitt): Definerer starttilstanden eller forutsetningene.

When (Når): Beskriver handlingen eller hendelsen som skjer.

Then (Så): Forventet resultat etter at handlingen er utført.

Et enkelt eksempel

Gherkin

Feature: Innlogging

Scenario: Vellykket innlogging med gyldig passord

Given brukeren er på innloggingssiden

When brukeren skriver inn gyldig e-post og passord

And klikker på logg inn-knappen

Then skal brukeren bli omdirigert til kontooversikten

Scrum

- Hva er Scrum?
- Hva legger Scrum fokus på?
- Hvordan er Scrum bygd opp?
- Hvilke seremonier består Scrum av, og hva omfatter de?

Systemutviklingsmetodikk
Smidige metoder

Tidsboksbasert *smidig metode* som ikke forteller oss hvordan vi skal utvikle koden eller modellere krav, men hvordan vi skal jobbe og få fremgang.

Fokus på kryssfunksjonalitet der utviklerteamet ikke har spesifikke roller

Utfører sprinter (tidsavgrensede iterasjoner) med demo mot slutten eller levert produkt

Seremonier/møter:

- **Sprint planning meeting:** Product owner og utviklingsteam møter og avklarer hva som skal inngå i kommende sprint. Henter ut ProductBackLogItems fra ProductBacklog, justerer disse og legger i SprintBacklog. PO bestemmer hvilke PBI. Populerer også scrum-board med oppgaver
- **Standup møte:**
 - **Sprint review:** Utviklerteam, PO og interessenter møter etter ferdig sprint og resultat presenteres. Tilbakemelding gis og evaluering av produkt skjer.
 - **Retrospektiv:** Utviklerteam innad reflekterer hva som gikk bra under spint og arbeidsprosess.

Lean

- Hva er Lean?
- Hvilke prinsipper består Lean av, og hva betyr de?

Systemutviklingsmetodikk
Metodikk

Lik smidig metode, men med mer vekt på å ikke lagre ting uten verdi.

Prinsipper:

- **Eliminate waste:** alt som produseres må ha verdi. Altså, unngå funksjoner kunden ikke vil ha selv om du liker det, dokumentasjon som er bare flesk, mye byråkrati osv
- **Build quality in:** God kvalitet på koden og systemet
- **Create knowledge:** Korte feedback looper mellom kunde og utvikler slik at flere ting kan testes og endringer kan gjøres hyppig.
- **Defer commitment:** Vent med handlinger som er irreversible til man har lært nok om konsekvensene av de handlingene
- **Deliver fast:** Tenk enkelt og unngå overengineering. Dette skaper unødvendig kompleksitet og administrering. Hovedfunksjonaliteten skal prioriteres og må ut i produksjon så raskt som mulig.
- **Respect people:** La utviklere ha ansvar over selve utviklingsprosessen og ledere fokuserer på sitt.
- **Optimize the whole:** Forstå helheten i et system, forstå hoveddomene og avhengigheter mellom komponenter

Kanban

- Hva er bakgrunnen for Kanban, og hvorfor er det det?
- Hva har Kanban fokus på?
- Hvordan er produktet levert?
- Hvordan er Kanban gjennomført?
- Hva tar Kanban i bruk?
- Hvordan er Kanban bygd opp, og hvordan er dette ulikt fra Scrum?
- Hva er formålet med Kanbans oppbygning?
- Hva er bivirkningen av Kanbans oppbygning?

Systemutviklingsmetodikk
Lean

Flytbasert: Fokus på å unngå flaskehalser under utvikling

Fokus på kontinuerlig flyt med fleksible prosesser og visuell styring uten faste iterasjoner.

Produkt levert kontinuerlig

Ingen standard møter

Bruker et **kanban-board**: Lik scrum-board bare at det er begrenset oppgaver på hver kolonne. Formål: Dersom man har lite oppgaver i kolonnen man jobber i kan man bidra i andres kolonner som har flere oppgaver. Disse kolonnene fungerer som en flaskehals som bremser det prosessen.

Domain-Driven Design

- Hva har Domain-Driven Design fokus på?
- Hva kjennetegner Domain-Driven Design?
- Hva er Domain-Driven Design godt egnet for?
- Hva er prioritert mest under utvikling ved Domain-Driven Design?

Systemutviklingsmetodikk
Lean

Høyt fokus på domenet

Koden skal gjenspeile virkeligheten, altså domenet

Godt egnet for skalering og endringer

Fokuserer på det viktigste i domenet

DevOps

- Hva er DevOps?
- Hva sikrer DevOps?
- Hva er forskjellen på Agile og DevOps, og hva er deres relasjon?
- Hva er CI?
- Hva er CD?
- Hva er de ulike DevOps verktøyene?

Systemutviklingsmetodikk
Metodikk

En kultur og praksis som kombinerer **utvikling (Dev)** og **drift (Ops)** for å muliggjøre kontinuerlig integrasjon og leveranse.

Agile vs DevOps: Agile er en metode (fokus på utvikling), DevOps er en kultur (fokus på leveranse til produksjon). Kombinert: «Agile DevOps»-kultur.

CI (Continuous Integration): Kode integreres hyppig i en felles gren. Endringer bygges og testes automatisk.

CD (Continuous Delivery/Deployment): Kodeendringer deploys automatisk til produksjon.

DevOps-verktøy:

Utvikling: GitLab/GitHub, Jira/Confluence

CI (bygg/test): Maven, Gradle, Jenkins, JUnit, Selenium

CD (deploy): Kubernetes, Helm, Docker

Drift/monitorering: Kibana, Grafana

REST

- Hva er REST?
- Hva er REST basert på?
- Hvordan blir data og funksjonalitet behandlet?
- Hvordan manipuleres det som blir behandlet?
- Hva er de ulike manipulerings metodene?

Systemutviklingsmetodikk
Metodikk

REpresentational State Transfer – arkitekturstil (ikke protokoll) for design av API-er.

- Basert på klient-tjener-paradigmet der klient og server utveksler representasjoner av ressurser
- Data og funksjonalitet behandles som **ressurser** tilgjengelig via **URI-er**
- Ressurser manipuleres med veldefinerte HTTP-operasjoner: GET, POST, PUT, DELETE
- Eksempel: GET /candidates (hent alle), POST /candidates (legg til ny), DELETE /candidates/{id} (slett)

Testing

Systemvalidering

Hvorfor teste?

- Hvorfor er det viktig å teste?
- Når tester man?

Testing
Systemvalidering

For å validere at systemet:

- Responderer riktig ved interaksjon/input
- Ikke har bugs
- Har god ytelse under edge cases
- Møter krav til brukervennlighet
- Møter kravene satt av product owner
- Kan installeres i riktig miljø
- Kan håndtere endringer

Tester vanligvis i faser i prosjektet, mens med smidig metode tester man kontinuerlig og automatisk gjennom DDD.

Tradisjonell testing

- Når skjer testingen i tradisjonell testing?
- Hvem lager testene i tradisjonell testing?
- Hva kjennetegner tradisjonell testing?
- Hva er fordelene og ulempene med tradisjonell testing?

Testing

Tradisjonell vs Smidig testing

Utvikling og testing skjer separat: ett team utvikler, et annet (QA-teamet) kvalitetssikrer

Produktet sendes tilbake til utvikling dersom krav ikke er oppfylt

Ulempe: mye tid brukt på kommunikasjon fremfor utvikling

Smidig testing

- Når skjer testingen i smidig testing?
- Hvem lager testene i smidig testing?
- Hva kjennetegner smidig testing?
- Hva er fordelene med smidig testing?

Testing

Tradisjonell vs Smidig testing

Når og hvem

Utviklere skriver og tester egen kode kontinuerlig

Kjennetegn

Funksjonalitet blir integrert og forbedret i små inkremitter

Fordeler

Kan bruke et QA-team for å validere viktige scenarioer

Bruker automatiserte tester

Automatiserte tester

- Når skjer testingen i automatiserte tester?
- Hva kjennetegner automatisert testing?
- Hva er fordelene og utfordringer med automatisert testing?

Testing

Tradisjonell vs Smidig testing

Når

Testkjøring trigges ved opprettelse av pull requests

Kjennetegn

Enhetstester og integrasjonstester kjøres automatisk i CI-pipelines

Fordeler

- **Rapporter:** Resultatrapport legges ved PR for rask feedback
- **Høyere hastighet:** Kjører gjentakende tester på brøkdelen av tiden det tar manuelt, noe som fremskynder hele leveringsprosessen.
- **Kontinuerlig kontroll (CI/CD):** Tester kan kjøres automatisk hver gang ny kode legges til, og fanger opp feil før de når brukerne.
- **Forutsigbar kvalitet:** Datamaskiner blir aldri slitne eller mister fokus, og leverer alltid konsekvente resultater.
- **Kostnadsbesparende over tid:** Reduserer behovet for kjedelig manuelt arbeid og gjør det billigere å vedlikeholde produktet.

Ulemper / Utfordringer

- **Høy oppstarts- og vedlikeholdskostnad:** Det krever tid og ressurser å skrive

selve testene, og de må oppdateres når programvaren endres.

- **Krever riktig fokus:** Dårlig designede tester som ikke tester kjernefunksjonaliteten gir liten eller ingen verdi

Egne test-/staging-miljøer

- Hva er Egne test-/staging-miljøer
- Hva kjennetegner Egne test-/staging-miljøer?
- Hva er fordelene med Egne test-/staging-miljøer?

Testing

Tradisjonell vs Smidig testing

Når og hvem

Kan teste hele applikasjonen før release – men må ikke

Kjennetegn

Testmiljøet speiler produksjonsmiljøet

Fordeler

Mer realistisk testing ettersom at test- og produksjonsmiljøet speiler

QA-teamet kan teste hele applikasjonen før release

Infrastructure-as-code (IaC) gjør oppsett og versjonshåndtering av miljøer enkelt

CI og små endringer

- Hva er fordelene med CI og små endringer?

Testing

Tradisjonell vs Smidig testing

Fordeler med CI og små endringer

Mindre diff mellom gammel og ny versjon

Enkel isolering og sporing av endringer

Redusert sannsynlighet for feil og enklere rollback ved problemer

Prioritering av testing

- Hva er viktig ved prioritering av testing?

Testing

Tradisjonell vs Smidig testing

Prioritering av testing

Finn riktig nivå på testdekning basert på risiko

Prioriter tester der de gir størst verdi og reduserer mest usikkerhet

Risk-poker

- Hva er Risk-poker?
- Hva brukes Risk-poker til?
- Hva er fordelene ved Risk-poker?

Testing
Systemvalidering

Brukes ved å beregne *risiko* og *konsekvens* av feil og av evt risikofaktorer. Alle på teamet "scorer" sannsynligheten og påvirkningen samtidig med et kortstokk, så bruker vi snittet. Siden hele teamet er med er treffsikkerheten større.

Test-data

- Hva er Test-data?
- Hvorfor bruker vi Test-data?
- Hvilken egenskap har Test-data?
- Hva er fordelene med Test-data?

Testing
Systemvalidering

Hva?

Testdata er et utvalg av informasjon eller datasett som brukes for å verifisere at dataprogrammer, applikasjoner eller IT-systemer fungerer som de skal

Hvorfor?

Når vi kjører tester er vi avhengig av et forutsigbart miljø for at programvaren ikke blir påvirket av eksterne faktorer, da dette kan gjøre det vanskelig å isolere feil.

Egenskap

Testdataene må derfor være det samme under testprosessen og prosessen må også være deterministisk.

Fordelene

Utviklere kan da vurdere resultatene testene mer grundig og effektivt.

Mocking

- Hva er Mocking?
- Når bør man bruke Mocking?
- Hvilke utviklingsmønstre passer godt til Mocking?
- Hvilken egenskap har Mocking som gjør det ideelt?
- Hva er fordelene med Mocking?

Testing
Systemvalidering

Hva?

Mocking er en teknikk der man erstatter ekte og komplekse deler av et system (som en database, et eksternt API eller en betalingstjeneste) med simulerte "jukse-objekter" (mocks).

Når?

De gangene vi ikke kan jobbe med reell data bør man bruke Mocking.

Utviklingsmønstre og egenskap

Ofte er klasser avhengig av hverandre som gjør det vanskelig å isolere delene fra hverandre og teste klassene hver for seg. Da er det lurt å bruke utviklingsmønstre som Dependency Injection for å sette moduler inn i hverandre under kjøring. Dette gjør at man kan bytte ut komponenter eller klasseinstanser med "mock" objekter som implementerer de samme metodene som instansene, men er deterministisk.

Fordelene

Slik blir det altså mulig å teste ut funksjonaliteten til én klasse om gangen basert på de hardkodede dataene returnert av mock-objektene hvor man vanligvis ikke

kunne skrevet automatiserte tester.

Testfaser

- Hva er de ulike testfasene?

Testing
Systemvalidering

Fase: *Beskrivelse*

- **Oppsett:** Setter opp testdataene og sørger for at de holder seg like hver gang testen kjøres. Oppretter altså en fasttilstand «test fixture»
- **Gjennomføring:** Utfører operasjonene spesifisert i testen
- **Verifisering:** Bekrefter at det som skulle skje har skjedd og verifiserer resultatene av testen vha f.eks. assert
- **Opprydding:** Dersom man benytter seg av eksterne ressurser er det viktig å rydde opp slik at vi får et identisk miljø på nytt til neste test

Enhetstesting

- Hva er Enhetstesting?
- Hva er fordelene med Enhetstesting?
- Hva er målet med Enhetstesting?

Testing
Test-typer

Hva?

Utviklertest av individuelle klasser på funksjonsnivå.

Fordelene

Automatisert og repeterbar

Kjøres alltid som en del av byggeprosessen

Mål

Identifisere feil tidlig slik at de kan bli rettes før integrasjons med andre deler av programvaren

Integrasjonstesting

- Hva er Integrasjonstesting?
- Hva er målet med Integrasjonstesting?
- Hvilke valg kan gjøres ved Integrasjonstesting?

Testing
Test-typer

Hva?

Tester samspillet og integrasjon mellom ulike komponenter som allerede er testet individuelt.

Mål

Oppdage feil i grensesnittet og samhandlingen.

Valg

Kan og bør automatiseres.

Systemtesting

- Hva er Systemtesting?
- Hva er fordelene med Systemtesting?
- Hva er målet med Systemtesting?
- Hvilke valg kan gjøres ved Systemtesting?

Testing
Test-typer

Hva?

Tester et fullt integrert system for å sjekke om systemet møter kravene.

- Testing av hele systemet
- Sjekker at alt fungerer sammen

Fordelene

- Finner feil mellom deler av systemet
- Gir trygghet før levering

Mål

- Sjekke at systemet oppfyller krav
- Finne feil før bruk

Valg

Kan bruke verktøy som Selenium, men siden vi er avhengig av alle ressurser (db, eksterne tjenester, API osv) kan dette bli vanskelig å gjennomføre da det er vanskelig å oppnå repeterbare tester.

Akseptansetesting

- Hva er Akseptansetesting?
- Hva er viktig med Akseptansetesting?
- Hva er fordelene med Akseptansetesting?

Testing
Test-typer

Hva?

Type systemtesting som utføres av kunden for å verifisere om systemet møter kravene i kontrakten.

Viktig

Akseptansetesting i BDD handler om automatisk testing av user-stories og kan være utviklet i samsvær med kunden.

Fordeler

Er manuell og automatisk

Regresjonstesting

- Hva er Regresjonstesting?
- Når skjer Regresjonstesting?
- Hva slags tester er Regresjonstester?
- Hvilke valg kan gjøres ved Regresjonstesting?

Testing
Test-typer

Hva?

Sikrer at systemet ikke brykker ved endringer der testen kjøres før og etter en endring.

Når?

Med kontinuerlig endring skjer dette implisitt siden testene kjøres ved innsjekk av kode.

Typer

Enhetstester fungerer derfor som regresjonstesting. Kan også lage regresjonstester på integrasjon- og akseptansenivå.

Valg

Regresjonstesting kan automatiseres siden man utfører nøyaktig samme test før og etter en endring.

Smoke-testing/Sanity-testing

- Hva er Smoke-testing/Sanity-testing?
- Når brukes Smoke-testing/Sanity-testing?
- Hva er forskjellen på Smoke-testing og Sanity-testing?
- Hvorfor er Smoke-testing og Sanity-testing nyttig?

Testing
Test-typer

Hva?

Ad hoc type test der man prøver ut de viktigste delene av et system.

Når?

Kan være greit å gjøre før systemet sendes videre til andre tester.

Forskjell og nyttinghet:

Smoke-testing: funker på kritisk funksjonalitet i systemet

Sanity-testing: fokus på ny funksjonalitet

Destruktiv testing

- Hva er destruktiv testing?
- Hvordan foregår Destruktiv testing?
- Hva kan ikke gjøres ved Destruktiv testing?
- Hva kreves for å utføre destruktiv testing?
- Hva er forskjellen på destruktiv og utforskende testing?

Testing
Test-typer

Hva og hvordan?

Type ad hoc, kreativ og utforskende testing hvor man prøver å få systemet til å feile.

Hva kan ikke gjøres?

Kan ikke automatiseres.

Krav?

Må bruke menneskelig kreativitet og erfaring.

Forskjell

- **Destruktiv:** Destruktiv testing, men black-box
- **Utforskende:** Destruktiv testing, men white-box

Usability testing

- Hva oppnåes ved Usability testing?
- Hva testes Usability testing?

Testing
Test-typer

Tester brukervennligheten: GUI og om kunden er fornøyd

Ytelsestesting

- Hva er ytelsestesting?
- Hvordan måler vi ytelse?
- Hva er flaskehalser?
- Hvordan finner vi flaskehalser?
- Hva er ofte årsakene til flaskehalser?
- Hva er de ulike typene med ytelsestesting?

Testing
Test-typer

Hva

- Hvor mye last tåler systemet
- Hvor mye plutselig last tåler systemet
- Ytelse over tid med tung arbeid
- Lekkasje?

Hvordan vi måler ytelse:

- Antall samtidige brukere
- Transaksjoner per tid
- Throughput
- Responstider

Flaskehalser

Flaskehalser i systemet er hvor systemet er treigest/svakest, og det som holder ytelse tilbake

Hvordan finner vi flaskehalser?

Kan finne flaskehalser ved å teste responstid på ulike steder og nivåer. Når flaskehalsen er funnet må vi finne årsaken.

Hva er ofte årsakene til flaskehalser?

Ofte er årsaken knyttet til CPU, minne, tråder, db tilkobling osv.

Typen ytelsestester

- **Last-Test:** Hvordan systemet oppfører seg ved forventet last. Hvor mye tåler systemet før ytelsen degraderes vesentlig. Hva er det som gjør at systemet degraderes
- **Stress test:** Finne systemets øverste limit. Når bryter systemet sammen og hva er årsaken?
- **Spike-test:** Hvordan systemet oppfører seg ved plutselig høy last.
- **Utholdenhetstest:** Hva skjer når systemet utsettes for jevn last over tid.

Black-box vs white box testing

- Hva er blackbox-testing?
- Hva bygger blackbox-testing på?
- Hva er whitebox-testing?
- Hva testes ved whitebox-testing?

Testing
Test-typer

Blackbox

Testing der testeren ikke har kjennskap til systemets interne kode eller oppbygning.

Testingen baseres på krav, spesifikasjoner og forventet oppførsel.

Whitebox

Testing der testeren har full kjennskap til systemets interne kode og struktur.

Testingen brukes for å kontrollere den interne oppbygningen og logikken i programvaren.

Automatisk eller manuell testing

- Hva er fordelene ved automatiske tester?
- Hva er de ulike automatiske test typene?
- Hva er ulempene ved automatiske tester?
- Hva er fordelene ved manuelle tester?
- Hva er de ulike manuelle test typene?
- Hva er de ulike test typene som er både manuelle og automatiske?

Testing
Test-typer

Automatisk

- **Fordeler:** Raske og nøyaktige.
- **Test typer:** Enhetstester og regresjonstest
- **Ulemper:** Finner ikke noe eksplisitt

Manuelle tester

- **Fordeler:** Kreative og finne fort vanlige feil
- **Test typer:** utforskende, destruktiv testing

Begge

- **Test typer:** Smoke, sanity, usability, integrasjon, akseptanse, ytelsestesting og systemtesting

Testing principles

- Hva er de ulike testing prinsippene?

Testing
Test-typer

1. Testing shows the presence of defects
2. Exhaustive testing is impossible
3. Early testing
4. Defects clustering
5. Pesticide paradox
6. Testing is context dependent
7. Absence of error fallacy

Profesjonsetikk

Ansvar og etikk

GDPR

- Hva står GDPR for?
- Hva er formålet med GDPR?
- Hva er de grunnleggende prinsippene i GDPR?
- Hva er de registrertes rettigheter?
- Hvilke unntak finnes det ved disse retningslinjene?

Profesjonsetikk
Ansvar og etikk

Hva står GDPR for?

General Data Protection Regulation

Formål

EUs personvernordning innført i norsk lov via personopplysningsloven. Regulerer all behandling av personopplysninger - fra innsamling og lagring til sletting og utlevering - for å beskytte individets rett til privatliv og kontroll over egen data.

Grunnleggende prinsipper

- **Lovlighet, rimelig og gjennomsiktighet** - .
- **Formålsbegrensning** - opplysninger samles inn kun for angitte mål.
- **Dataminimering** - kun det som er nødvendig for formålet skal innhentes.
- **Riktighet** - feil data skal rettes.
- **Lagringsbegrensning** - data skal ikke lagres lengre enn nødvendig.
- **Integritet og konfidensialitet** - tiltak skal hindre uautorisert tilgang, tap eller endring.
- **Ansvarliggjøring** - behandlingsansvarlige må kunne sikre at prinsippene følges.

De registreters rettigheter

- **Rett til informasjon:** vite hva som samles og hvorfor
- **Rett til insyn:** tilgang til sine egne opplysninger
- **Rett til retting:** korrigerer av uriktige data
- **Rett til sletting:** kreve fjerning av unødvendige data
- **Rett til begrensning:** stoppe videre bruk av data i visse situasjoner
- **Rett til å protestere:** nekte automatisk eller markedsføringsbasert behandling
- **Rett til dataportabilitet:** motta egne data i maskinlesbart format og overføre dem

Unntak:

Noen rettigheter kan begrenses, f.eks. når behandlingen er nødvendig for juridisk forpliktelse, ivareta offentlig sikkerhet eller forskning. Behandlingsansvarlige som må bevise at unntaket gjelder.

Profesjonsetikk

- Hva er profesjonsetikk?
- Hva er ingeniøretikk?
- Hva er det ingeniørens ansvar omfatter?
- ...

Profesjonsetikk
Ansvar og etikk

Hva er profesjonsetikk?

Omfatter de moralske normene og forventningene som knytter seg til et bestemt yrke.

Ingeniøretikk

Etiske problemstillinger oppstår når teknologiske løsninger kan ha utilsiktede eller skadelige konsekvenser.

Ingeniørens ansvar omfatter blant annet:

- **Sosialt ansvar:** Teknologi kan forandre arbeidsmarkedet, påvirke demokratier eller krenke kulturelle normer.
- **Sikkerhet og pålitelighet:** Eks – selvkjørende biler kan aldri bli feilfrie; hvor legges terskelen for trygghet?
- **Brukeres velferd:** Automatisering som erstatter menneskelig arbeidskraft – hvilken rolle spiller utvikleren i å dempe sosiale omkostninger?
- **AI og militær bruk:** Droner med dødelig kraft reiser spørsmål om hvem som har siste etiske ansvar

Kodeverk og retningslinjer:

- **NITO:**
 - Dataingeniøren skal vise respekt for kolleger uavhengig av bakgrunn
 - vedkjenne seg faglig ansvar
 - følge anerkjente kvalitetsnormer,
 - bidra til åpenhet om teknologiske konsekvenser.
- **IEEE/ACM Code of Ethics:** Internasjonale etiske retningslinjer for utvikling av IT løsninger.
 - SAMFUNN - Programvareingeniører skal handle konsekvent i offentlig interesse.
 - KUNDE OG ARBEIDSGIVER - Programvareingeniører skal handle på en måte som er til beste for kunde og arbeidsgiver og i samsvar med offentlige interesser.
 - PRODUKT - Programvareingeniører skal sikre at deres produkter best mulig oppfyller profesjonelle standarder.
 - VURDERING - Programvareingeniører skal utvise faglig skjønn og opprettholde integritet og uavhengighet.
 - LEDELSE – Ledere skal fremme en etisk tilnærming til styring av programvareutvikling og vedlikehold.
 - PROFESJON - Programvareingeniører skal fremme integriteten og omdømmet til profesjon i samsvar med offentlig interesse.
 - KOLLEAGER - Programvareingeniører skal være rettferdige mot og støtte sine kollegaer.
 - IDENTITET - Programvareingeniører skal delta i livslang læring innen praksis innen sin profesjon og skal fremme en etisk tilnærming til utøvelsen av yrket.

Nøkkelprinsipper i profesjonsetikken:

- **Ærlighet og integritet:** Vær åpen om begrensninger og mulige risikoer i egne løsninger.
- **Respekt for personvern:** Bygg inn personvernbeskyttelse fra starten («privacy by design»).
- **Kvalitetsbevissthet:** Følg anerkjente utviklings- og teststandarder for å unngå feil.
- **Ansvar for konsekvenser:** Identifiser og håndter etiske dilemmaer proaktivt, ikke først når problemer oppstår.
- **Kontinuerlig vurdering:** Etikkarbeid er iterativt - reflekter jevnlig over nye teknologiske muligheter og følg ny veiledning fra faglige organisasjoner.

Hvorfor det henger sammen med kvalitet:

Programvare som tar hensyn til etiske krav er mer robust, unngår alvorlige sikkerhets- og personvernbrudd, gir høyere brukertilfredshet og beskytter utviklerens og organisasjonens omdømme. Manglende etisk forankring kan gi

programvare av dårlig kvalitet, for eksempel systemer med innebygde skjevheter (bias), mangelfull sikkerhet eller løsninger som bryter brukernes tillit og fører til omdømmeskade.

SuSAD

Prosjektplanlegging

Planlegging og gjennomføring

Prosjektplanlegging

- ...

Prosjektplanlegging
Planlegging og gjennomføring

Visiondokument

Beskriver prosjektets formål, omfang og hensikt.

I smidige metoder er visiondokumentet mer fleksibel og mindre deltaljert og kan oppdateres.

Gir felles forståelse av prosjektet blant interessenter (prosjektteamet, ledelse, aksjonærer, kunde, osv)

Gir en retning for arbeidet: formål, målgruppe, funksjonelle og ikke-funksjonelle krav

Identifisering av interessenter og brukerbehov

Kommunikasjon med prosjektets interessenter

Risikoanalyse

Hensikt

Identifisere potensielle risikoer som kan påvirke prosjektets fremgang

Prioritering av risiko

Tiltak

Forbedring av beslutningsprosess

Øke prosjektets robusthet og gi prosjektteamet og interessentene en bedre forståelse av potensielle trusler og utfordringer underveis i prosessen.

